

함수 이해를 돕는 20문제

Python 3 함수 | 기초 개념부터 입력값 처리까지

각 문항은 Python 3에서 독립적으로 실행한다. 출력은 print()가 표시한 값만 적는다.

01 함수는 왜 사용할까?

객관식 · 기초

파이썬 함수에 대한 설명으로 가장 적절한 것을 고르시오.

- (1) 함수는 프로그램에서 한 번만 사용할 수 있다.
- (2) 작업을 하나로 묶어 필요할 때 호출하고 재사용할 수 있다.
- (3) 모든 함수에는 반드시 하나 이상의 인수가 필요하다.
- (4) def로 함수를 정의하면 본문의 코드가 즉시 실행된다.

정답 (2)

해설 함수는 특정 작업을 수행하는 코드를 묶어 필요할 때 실행하는 단위이다. 같은 함수를 여러 번 호출하면 중복 코드를 줄일 수 있다. 인수가 없는 함수도 만들 수 있으며, 아래 문항처럼 def로 함수를 정의하는 것만으로 본문이 실행되지는 않는다.

02 함수의 정의와 호출 구분하기

실행 결과 · 기초

다음 코드의 출력 결과를 줄 순서대로 쓰고, 함수 본문이 실행된 횟수를 구하시오.

```
def say_hello():  
    print("Hello")  
  
say_hello()  
say_hello()
```

정답 Hello가 두 줄 출력된다. 함수 본문은 총 2번 실행된다.

해설 def는 함수를 정의하는 키워드이다. 첫 줄 끝에는 콜론(:)을 쓰고, 함수 본문은 들여쓰기로 구분한다. say_hello()를 호출할 때마다 본문의 print()가 실행되므로 Hello가 한 번씩, 총 두 번 출력된다.

매개변수와 반환값

03-04번 | 입력한 값이 계산 결과가 되는 과정

03 매개변수와 인수 구분하기

단답형 · 기초

함수 이름, 매개변수, 호출할 때 전달한 인수, 최종 출력값을 각각 쓰시오.

```
def area(width, height):  
    return width * height  
  
print(area(4, 6))
```

정답 함수 이름: area / 매개변수: width, height / 인수: 4, 6 / 출력값: 24

해설 매개변수는 함수 정의에서 입력값을 받는 이름이고, 인수는 호출할 때 실제로 전달하는 값이다. 이 호출에서는 width에 4, height에 6이 전달된다. 함수가 4 * 6의 결과인 24를 반환하면 print()가 그 값을 출력한다.

04 반환값을 변수에 저장하기

빈칸 완성 · 기초

add 함수를 한 번 호출하여 3과 5의 합을 result에 저장하도록 빈칸을 채우고, 출력값을 구하시오.

```
def add(a, b):  
    return a + b  
  
result = ____  
print(result * 2)
```

정답 빈칸: add(3, 5) / 출력값: 16

해설 add(3, 5)가 반환하는 값은 8이므로 result에 8이 저장된다. 마지막 줄에서 result * 2를 계산하므로 16이 출력된다. return으로 받은 계산 결과는 변수에 저장한 뒤 다른 연산에서도 사용할 수 있다.

출력과 반환, 실행의 종료

05-06번 | print()와 return의 역할 구분

05 출력과 반환은 어떻게 다를까?

오류 수정 · 기초

원래 코드의 출력 결과를 쓰시오. 이어서 square의 본문 한 줄을 수정하여 마지막 줄에서 25만 출력되도록 하시오.

```
def square(n):  
    print(n * n)  
  
value = square(5)  
print(value)
```

정답 원래 출력: 25, None 순서 / 수정할 한 줄은 다음과 같다.

```
return n * n
```

해설 print()는 값을 화면에 표시하지만 그 값을 함수의 계산 결과로 돌려주지는 않는다. 원래 square는 반환문 없이 끝나므로 None을 반환하고, value에도 None이 저장된다. 본문을 return으로 바꾸면 25가 저장되어 마지막 줄에서 한번만 출력된다.

06 return 이후의 실행 흐름

실행 결과 · 기초

출력 결과를 줄 순서대로 쓰고, print("B")가 실행되는지 설명하시오.

```
def check():  
    print("A")  
    return 10  
    print("B")  
  
print(check())
```

정답 A, 10이 차례로 출력된다. B는 출력되지 않는다.

해설 check()를 호출하면 먼저 A를 출력한 뒤 return 10을 만난다. 이 반환문에서 해당 함수 호출의 실행이 끝나므로 뒤의 B 출력문은 실행되지 않는다. 함수가 돌려준 10은 바깥의 print()가 출력한다.

변수의 범위와 값의 전달

07-08번 | 함수 안에서 바뀌는 값 확인하기

07 함수 안과 밖의 변수

실행 결과 · 이해

두 출력값을 순서대로 쓰고, 이름이 같은 두 변수가 서로 어떤 관계인지 설명하시오.

```
value = 10
def show():
    value = 3
    print(value)

show()
print(value)
```

정답 3, 10이 차례로 출력된다. 두 value는 서로 다른 범위의 변수이다.

해설 이 코드에서 함수 안의 value = 3은 지역 변수 value에 값을 대입한다. show() 안에서는 이 지역 변수를 사용하므로 3이 출력된다. 함수 밖의 value는 변경되지 않으므로, 호출 후 마지막 줄에서는 10이 출력된다.

08 정수 매개변수에 다시 대입하기

실행 결과 · 이해

두 출력값을 순서대로 쓰고, increase() 호출 후에도 number의 값이 유지되는 이유를 설명하시오.

```
number = 5
def increase(n):
    n = n + 1
    return n

print(increase(number))
print(number)
```

정답 6, 5가 차례로 출력된다.

해설 함수가 호출되면 n도 number가 가리키는 정수 5를 가리킨다. 정수는 변경할 수 없는 객체이므로 $n = n + 1$ 은 n이 계산 결과인 6을 가리키도록 바꾸는 동작이다. 바깥의 number가 가리키는 값은 바뀌지 않아 5로 유지된다.

기본값과 조건에 따른 반환

09-10번 | 입력 조건에 맞게 동작하는 함수

09 매개변수의 기본값 활용하기

실행 결과 · 이해

다음 두 함수 호출의 출력값을 순서대로 쓰시오.

```
def multiply(value, times=2):  
    return value * times  
  
print(multiply(3))  
print(multiply(3, 4))
```

정답 6, 12가 차례로 출력된다.

해설 times = 2는 times에 해당하는 인수를 생략했을 때 사용할 기본값이다. multiply(3)은 times에 2를 사용하므로 6을 반환한다. multiply(3, 4)는 직접 전달한 4를 사용하므로 12를 반환한다.

10 조건에 따라 다른 값 반환하기

빈칸 완성 · 이해

점수가 60 이상이면 "PASS", 그보다 낮으면 "RETRY"를 반환하도록 빈칸을 채우고, 두 출력값을 쓰시오. 점수는 0~100의 정수이다.

```
def judge(score):  
    if ____:  
        return "PASS"  
    return "RETRY"  
  
print(judge(60))  
print(judge(59))
```

정답 빈칸: score >= 60 / 출력: PASS, RETRY 순서

해설 60점도 통과 조건에 포함되므로 >가 아니라 >=를 사용한다. 60을 전달하면 if문 안의 return에서 PASS를 반환한다. 59를 전달하면 조건이 거짓이므로 마지막 return에서 RETRY를 반환한다.

반복과 반환 경로 확인

11-12번 | 들여쓰기와 return의 위치 확인하기

11 반복문을 포함한 함수

실행 결과 · 이해

출력값을 구하고, 반복문이 한 번 끝날 때마다 total의 값이 어떻게 바뀌는지 쓰시오.

```
def sum_to(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i  
    return total  
  
print(sum_to(4))
```

정답 출력값: 10 / 각 반복 직후 total: 1, 3, 6, 10

해설 range(1, n + 1)은 1부터 n까지의 정수를 차례로 제공한다. n이 4이므로 total에 1, 2, 3, 4를 더하면 값이 1, 3, 6, 10으로 변한다. return은 for문과 같은 들여쓰기 수준에 있어 반복이 모두 끝난 뒤 최종 합계 10을 반환한다.

12 반환값이 빠진 경로 찾기

오류 수정 · 이해

원래 코드의 출력 결과를 쓰시오. 이어서 정수 n이 짝수이면 True, 홀수이면 False를 반환하도록 한 줄을 추가하시오.

```
def is_even(n):  
    if n % 2 == 0:  
        return True  
  
print(is_even(4))  
print(is_even(5))
```

정답 원래 출력: True, None 순서 / if문 뒤에 같은 들여쓰기 수준으로 한 줄을 추가한다.

```
    return False
```

해설 n % 2는 n을 2로 나눈 나머지이다. 4는 조건을 만족해 True를 반환하지만, 5는 원래 코드에서 반환문을 만나지 않아 None을 반환한다. if문 밖에 return False를 두면 홀수 입력에도 명확한 불리언 값을 반환한다.

함수 객체와 lambda

13-14번 | 함수를 변수로 가리키고 간단히 표현하기

13 함수를 다른 변수로 가리키기

빈칸 완성 · 이해

operation이 subtract 함수 자체를 가리키도록 빈칸을 채우고, 출력값을 구하시오.

```
def subtract(a, b):  
    return a - b  
  
operation = ____  
print(operation(9, 4))
```

정답 빈칸: subtract / 출력값: 5

해설 파이썬에서 함수도 객체이므로 다른 변수에 대입할 수 있다. operation = subtract는 함수를 즉시 실행하는 문장이 아니다. 이후 operation(9, 4)를 호출하면 subtract의 본문이 실행되어 9 - 4의 결과인 5를 반환한다.

14 lambda로 간단한 함수 만들기

코드 작성 · 이해

아래 double과 같은 동작을 하는 함수를 lambda를 사용해 한 줄로 작성하시오. 함수는 double이라는 변수에 저장하고, double(7)의 출력값도 쓰시오.

```
def double(n):  
    return n * 2  
  
print(double(7))
```

정답 출력값: 14 / 함수 작성 예시는 다음과 같다.

```
double = lambda n: n * 2
```

해설 lambda 매개변수: 식 형태로 간단한 함수를 만들 수 있다. 콜론 뒤 식의 계산 결과가 반환되므로 return을 쓰지 않는다. n에 7을 전달하면 $n * 2$ 의 결과인 14를 반환한다.

함수 연결과 재귀 호출

15-16번 | 호출 순서와 종료 조건 이해하기

15 함수의 반환값을 다른 함수에 전달하기

실행 결과 · 응용

출력값을 구하고, add_one과 double이 어떤 순서로 실행되는지 설명하시오.

```
def double(n):  
    return n * 2  
  
def add_one(n):  
    return n + 1  
  
print(add_one(double(add_one(4))))
```

정답 출력값: 11 / 실행 순서: add_one(4), double(5), add_one(10)

해설 double에 전달할 인수를 구하려면 안쪽 add_one(4)를 먼저 실행해야 하므로 5를 얻는다. double(5)는 10을 반환하고, 가장 바깥의 add_one(10)은 11을 반환한다. 각 함수의 반환값이 다음 함수 호출의 인수가 된다.

16 재귀 함수의 종료 조건

실행 결과 · 도전

n이 0 이상의 정수일 때 출력값을 구하고, if n == 0 조건이 필요한 이유를 설명하시오.

```
def sum_down(n):  
    if n == 0:  
        return 0  
    return n + sum_down(n - 1)  
  
print(sum_down(3))
```

정답 출력값: 6 / n이 0일 때 0을 반환하여 재귀 호출을 멈춘다.

해설 재귀 함수는 자기 자신을 다시 호출하는 함수이다. sum_down(3)은 3 + sum_down(2), 이어서 3 + 2 + sum_down(1), 마지막으로 3 + 2 + 1 + 0을 계산한다. 종료 조건을 없애면 호출이 반복되다가 파이썬의 재귀 깊이 제한을 넘어 RecursionError가 발생한다.

콜백 함수와 리스트 처리

17-18번 | 다른 함수에 함수를 전달하는 방법

17 함수를 인수로 전달하기

실행 결과 · 응용

출력값을 구하고, apply 함수의 매개변수 fn에 무엇이 전달되는지 설명하시오.

```
def apply(value, fn):  
    return fn(value)  
  
def triple(n):  
    return n * 3  
  
print(apply(5, triple))
```

정답 출력값: 15 / fn에는 triple 함수 자체가 전달된다.

해설 apply(5, triple)에서 value는 5이고, fn은 triple 함수를 가리킨다. apply 안의 fn(value)가 triple(5)를 실행하므로 15를 반환한다. 이처럼 다른 함수에 인수로 전달되어 그 함수 안에서 호출되는 함수를 콜백 함수라고 한다.

18 map과 lambda 활용하기

실행 결과 · 응용

두 출력 결과를 순서대로 쓰시오. map은 각 원소에 함수를 적용한 결과를 차례로 얻는 데 사용하며, list()로 그 결과를 리스트로 만들 수 있다.

```
values = [1, 3, 5]  
result = list(map(lambda n: n * 2, values))  
print(result)  
print(values)
```

정답 [2, 6, 10], [1, 3, 5]가 차례로 출력된다.

해설 lambda n: n * 2에 1, 3, 5가 각각 전달되어 2, 6, 10을 반환한다. Python 3의 map()은 리스트가 아닌 반복자(iterator)를 반환하므로 list()로 결과를 모은다. 이 코드에서는 values를 변경하지 않아 원본 리스트가 그대로 유지된다.

여러 반환값과 사용자 입력

19-20번 | 튜플과 입력값 변환 활용하기

19 여러 계산 결과를 함께 반환하기

실행 결과 · 응용

출력값을 줄 순서대로 쓰고, return 뒤의 두 값이 quotient와 remainder에 각각 저장되는 과정을 설명하시오. a는 0 이상의 정수, b는 양의 정수이다.

```
def divide(a, b):  
    return a // b, a % b  
  
quotient, remainder = divide(17, 5)  
print(quotient)  
print(remainder)
```

정답 3, 2가 차례로 출력된다. 반환된 튜플 (3, 2)의 값이 두 변수에 나뉘어 저장된다.

해설 a // b는 정수 나눗셈의 몫, a % b는 나머지를 구한다. divide(17, 5)는 두 계산 결과를 튜플 (3, 2)로 묶어 반환한다. quotient, remainder = ...에서 튜플을 풀어 첫 번째 값 3은 quotient에, 두 번째 값 2는 remainder에 저장한다.

20 입력값을 계산하는 함수 완성하기

오류 수정 · 종합

input()에 2와 3을 차례로 입력한다. 실제 출력값과 그 원인을 쓰고, 마지막 줄의 add(a, b) 부분만 수정하여 정수의 합 5가 출력되도록 하시오.

```
def add(a, b):  
    return a + b  
  
a = input()  
b = input()  
print(add(a, b))
```

정답 원래 출력값: 23 / 수정: add(int(a), int(b))

해설 input()은 입력 내용을 문자열로 반환하므로 a와 b에는 "2", "3"이 저장된다. 문자열에 +를 사용하면 이어 붙여 "23"이 된다. int()로 두 값을 정수로 변환해 add에 전달하면 함수가 숫자의 합 5를 반환하고 print()가 이를 출력한다.